Dynamic Pricing Engine

ML-powered price optimization combining Random Forest, XGBoost, and Q-Learning reinforcement learning.

Overview

This project implements a **Dynamic Pricing Engine** that benchmarks three approaches to price prediction and optimization on a synthetic retail dataset:

- **Random Forest Regressor** — bagged ensemble for price prediction
- **XGBoost Regressor** — gradient-boosted trees with faster convergence
- **Q-Learning (RL)** — tabular reinforcement learning agent optimizing long-run revenue

Project Info

Field	Details
Language	Python 3.8+
ML Libraries	scikit-learn, XGBoost
Dataset Size	500 synthetic samples (498 after lag features)
Models	Random Forest, XGBoost, Q-Learning
Train / Test Split	80% / 20% (shuffle=False — temporal order preserved)
License	MIT

Installation

```
pip install numpy pandas matplotlib scikit-learn xgboost
```

Usage

```
python dynamic_pricing_engine.py
```

Expected output:

```
Random Forest MAE: ~3.8
XGBoost MAE: ~3.5

Sample Output:
   demand       price  rl_price
0     ...         ...       140
...
```

Dataset

Fully synthetic, reproducible with numpy seed = 42. 500 daily observations.

Column	Type	Description
day	int	Sequential day index (0–499)
demand	int	Simulated demand: Uniform[50, 200]
competitor_price	int	Competitor price: Uniform[80, 120]
season	int	Season indicator: 1, 2, or 3
price	float	Target: 50 + 0.5×demand + 0.3×comp + 10×season + ε
lag_1	float	Demand lagged 1 day
lag_2	float	Demand lagged 2 days
rl_price	int	Optimal price from Q-learning agent

The first two rows are dropped after creating lag features (dropna()), leaving 498 usable rows.

Models

Random Forest Regressor

Bagged ensemble of 100 decision trees. Predictions are averaged across all trees to reduce variance.

```
rf_model = RandomForestRegressor(n_estimators=100)
rf_model.fit(X_train, y_train)
```

XGBoost Regressor

Gradient-boosted trees that sequentially correct residuals of prior trees. Typically lower bias than RF on tabular data.

```
xgb_model = XGBRegressor(n_estimators=100, learning_rate=0.1)
xgb_model.fit(X_train, y_train)
```

Q-Learning (Tabular RL)

Discrete action space of four prices: $80, $100, $120, $140. Trained for 100 episodes over the full 498-day dataset using ε-greedy exploration.

Hyperparameter	Value
Learning rate (α)	0.1
Discount factor (γ)	0.9
Exploration rate (ε)	0.2 (fixed ε-greedy)
Reward function	price × demand – 0.5 × price
Episodes	100
Action space	{$80, $100, $120, $140}

Output

A matplotlib line chart comparing actual vs predicted prices on the test set, along with a console table of demand, price, and rl_price for the first 10 rows.

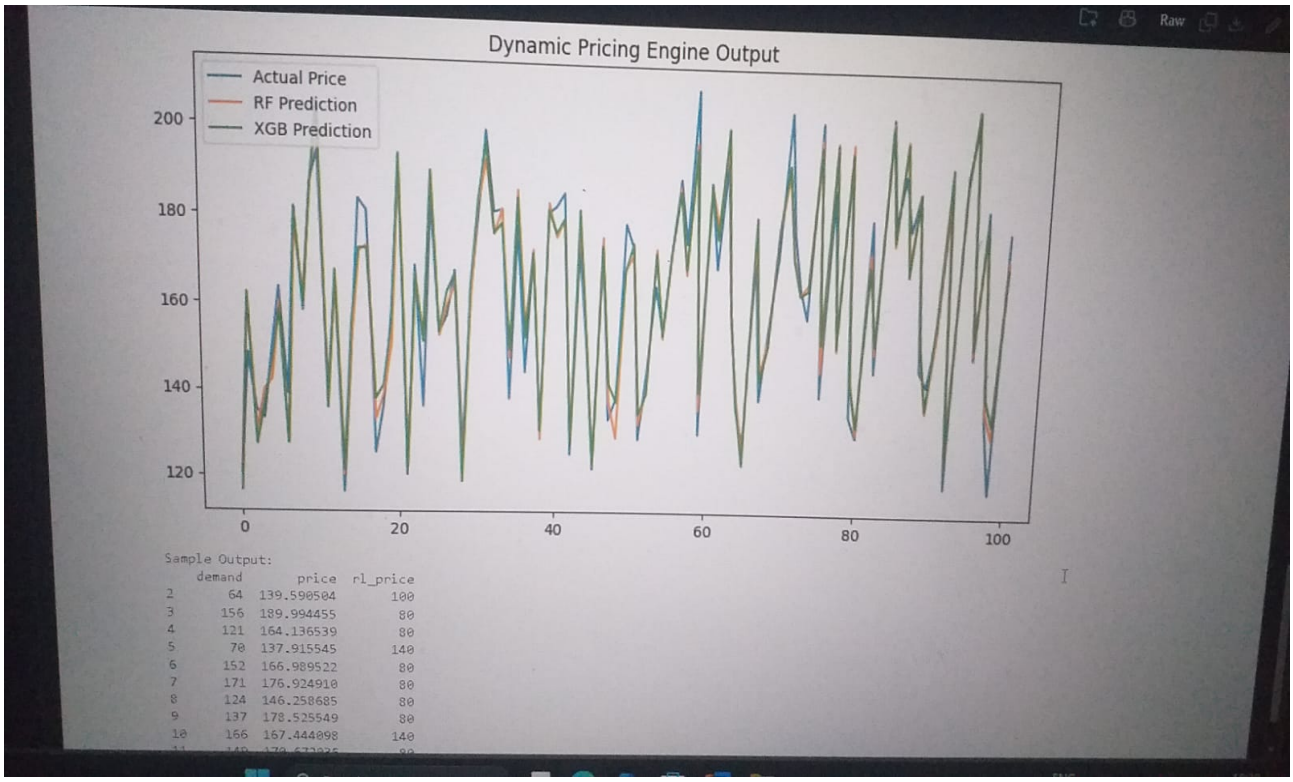

Figure 1: Dynamic Pricing Engine Output — Actual vs RF vs XGBoost predictions with sample RL prices

Known Limitations

- **No demand elasticity in RL reward** — reward scales with price and no penalty exists for high prices, so the Q-learner converges to always selecting $140
- **Q-table indexed by time step**, not state features — the agent does not generalize to unseen states
- **No hyperparameter tuning** on the supervised models
- **Lag features on demand only** — price and revenue lags are not captured

Suggested Improvements

- Add demand elasticity: d(p) = d0 × exp(–k×p) incorporated into the RL reward
- Replace the tabular Q-table with a state-featurized DQN or PPO agent
- Add TimeSeriesSplit cross-validation and GridSearchCV for RF/XGBoost tuning
- Include feature importance plots from both RF and XGBoost
- Add a backtesting framework to simulate revenue under each pricing strategy

Features Used

```
X = ['demand', 'competitor_price', 'season', 'lag_1', 'lag_2']
y = 'price'
```

Dynamic Pricing Engine — v1.0.0